

Privacy-Preserving Network Anomaly Detection

Detecting abnormal network behavior from encrypted
traffic metadata, without inspecting packet payloads

*Project report - what the system is, why it exists,
and how it works*

Dataset: UNSW-NB15 (official train/test split)

Models: Majority Baseline, Logistic Regression, Random Forest

Feature sets: FULL_METADATA, RESTRICTED_METADATA

Repository: github.com/TMStroo/Privacy-Preserving-Network-Anomaly-Detection

1. What this project is

This project is a machine learning experiment for detecting abnormal network behavior using only traffic metadata. Metadata means the observable properties of a communication flow: how long it lasted, how many packets and bytes it carried, timing statistics, protocol and connection state. It never looks at what the packets contain.

The practical motivation is encryption. Modern networks encrypt their payloads with TLS 1.3, QUIC and encrypted DNS, so the old approach of reading packet contents (deep packet inspection) either fails or requires breaking encryption, which is both hard and invasive. Network defenders still need to spot scanning, denial of service, exploits and other abnormal activity. The question this project asks is whether the remaining visible surface - the metadata - carries enough signal to do that.

Concretely, the system downloads the UNSW-NB15 intrusion detection dataset, validates it, applies an explicit feature policy that forbids payload and identity fields, trains three baseline classifiers under two feature regimes, evaluates them on the official held-out test set, and writes metrics and figures to disk. Everything is driven by one command and one configuration file so the experiment can be repeated exactly.

Research questions

- Primary: can encrypted network traffic metadata provide enough information to detect abnormal network behavior without inspecting message contents?
- Secondary: how much does detection performance change when the model is restricted to a small set of traffic metadata features?

Hypothesis

Traffic metadata contains sufficient signal to separate anomalous from normal traffic without payload inspection, and shrinking from the full metadata set to a small core subset will reduce performance measurably but not catastrophically - implying that a handful of flow statistics carry most of the discriminative information.

2. Why the problem matters

Encryption adoption is now near-universal on the public internet. That is good for privacy and bad for any defense that depends on reading content. At the same time, enterprises and operators cannot simply stop monitoring: scanning, command-and-control traffic, denial of service and data exfiltration all leave traces in how traffic behaves, not only in what it says.

There is also a legitimacy angle. Inspecting payload means touching user content, which raises legal and trust issues and is prohibited in some jurisdictions and networks. A detector that only consumes flow records (NetFlow/IPFIX) or packet headers can be deployed on links where content inspection is

not allowed. This project is a reproducible baseline for that idea: it measures exactly how much detection power survives when content is off the table.

Threat model

- Attacker: generates malicious traffic; may encrypt, obfuscate or mimic legitimate flows.
- Defender: observes only metadata - flow records, packet headers, timing, connection state. Cannot decrypt payloads.
- Assumption: the attacker cannot perfectly replicate the statistical profile of legitimate traffic across every metadata dimension at once.

3. Dataset

UNSW-NB15 (2015) was created by the Australian Centre for Cyber Security. It mixes real normal traffic with synthetic attack traffic covering denial of service, exploits, fuzzers, reconnaissance, shellcode, worms, backdoors and generic attacks.

- Training set: about 175,000 rows (official split).
- Test set: about 82,000 rows (official split, held out).
- 49 columns: 45 predictive fields, plus the binary label and the attack category.
- The label is converted to a clean binary target: 0 = normal, 1 = anomalous. The attack category column is never used as a feature.

The official split is used as-is. Rows are never randomly re-mixed between train and test, because that would leak information and inflate scores. The repository does not commit any dataset files: raw and processed data are gitignored, and a script generates small sample files so tests and CI run without the 250 MB download.

Current results in this report were produced on the bundled 100-row sample data, so the pipeline can be demonstrated anywhere. They are a smoke test, not a benchmark claim. Re-run on the full dataset for real numbers.

4. Feature policy: what the model may see

The privacy claim of this project is only as good as its feature list, so the policy is written in code and configuration rather than left implicit. Every column used for prediction is documented in `src/features/build_features.py` with a note on why it qualifies as metadata.

Used (metadata only)

- Flow timing: `dur`, `sinpkt`, `dinpkt`, `sjit`, `djit`, `tcprtt`, `synack`, `ackdat`
- Volume and rate: `spkts`, `dpkts`, `sbytes`, `dbytes`, `rate`, `sload`, `dload`
- TCP state and windows: `sttl`, `dttl`, `swin`, `dwin`, `stcpb`, `dtcpb`, `sloss`, `dloss`

- Packet size statistics: smean, dmean, trans_depth, response_body_len
- Behavioral aggregates: ct_srv_src, ct_state_ttl, ct_dst_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm, ct_src_ltm, ct_srv_dst, is_sm_ips_ports, is_ftp_login, ct_ftp_cmd, ct_flw_http_mthd
- Protocol, service and connection state: proto, service, state

Deliberately excluded

- srcip, dstip - IP addresses are identity-bearing and do not generalize.
- sport, dsport - ports can identify services and endpoints.
- Ltime, Stime - absolute timestamps enable correlation and tracking.
- attack_cat - it is a label, not a predictive feature.
- Anything derived from packet content - the whole point of the policy.

The two feature sets

Feature set	Transformed size	Contents
FULL_METADATA	37 columns	All policy-compliant metadata
RESTRICTED_METADATA	12 columns	Basic flow statistics and timing only

Both lists live in configs/config.yaml, so experiments can change features without touching code. The restricted set is dur, spkts, dpkts, sbytes, dbytes, sttl, dttl, sload, dload, tcprtt, synack, ackdat - nothing but sizes, counts and timing.

5. How the system works

The pipeline runs five stages in order, all orchestrated by `src/pipeline.py`:

Stage 1 - Ingest and validate

The download module fetches the UNSW-NB15 files into `data/raw` (or verifies them if already present). The validator then checks row and column structure, required columns, missing values, duplicate rows and the class balance of the target, and reports any issue before training starts.

Stage 2 - Clean and transform

Cleaning is documented and deterministic: numeric missing values are filled with the training median, categorical missing values with the mode, and rows with invalid target values are dropped. The target becomes 0/1. Categorical columns (proto, service, state) are one-hot encoded, numeric columns are standardized.

The scaler and encoder are fit on the training data only, then applied to the test set. No test statistic ever leaks into preprocessing.

Stage 3 - Train baselines

Three models are trained for each feature set with a fixed random seed of 42, wrapped in scikit-learn pipelines so preprocessing and fitting stay reproducible:

Model	Configuration
Majority Baseline	DummyClassifier, most_frequent
Logistic Regression	C=1.0, max_iter=1000, lbfgs, class_weight=balanced
Random Forest	n_estimators=200, max_depth=15, min_samples_split=5, class_weight=balanced

Stage 4 - Evaluate

Every model is scored on the official test set with precision, recall, F1, false positive rate, false negative rate, confusion matrix and class counts, plus ROC AUC and PR AUC when probabilities are available. Results are written as machine-readable JSON and CSV under `results/metrics/`.

Stage 5 - Plot

Figures are rendered to `results/figures/`: a confusion matrix for each main model, a model comparison by F1, a model comparison by false positive rate, the Random Forest feature importance chart, and the class distribution of train versus test.

Experiment matrix

The main experiment is a 2 x 3 grid: two feature sets crossed with three models, five real runs (the majority baseline needs only one, since it ignores features, but it is reported for both sets for symmetry). Train/test separation follows the dataset's official split throughout.

6. Results

All numbers below are read directly from results/metrics/evaluation_results.json at generation time.
Test set: 100 rows (47 normal, 53 anomalous) of sample data.

Feature Set	Model	Precision	Recall	F1	FPR	FNR
FULL_METADATA	Majority Baseline	0.5300	1.0000	0.6928	1.0000	0.0000
FULL_METADATA	Logistic Regression	0.6250	0.5660	0.5941	0.3830	0.4340
FULL_METADATA	Random Forest	0.9630	0.9811	0.9720	0.0426	0.0189
RESTRICTED_METADATA	Majority Baseline	0.5300	1.0000	0.6928	1.0000	0.0000
RESTRICTED_METADATA	Logistic Regression	0.6250	0.5660	0.5941	0.3830	0.4340
RESTRICTED_METADATA	Random Forest	0.9630	0.9811	0.9720	0.0426	0.0189

Reading the table

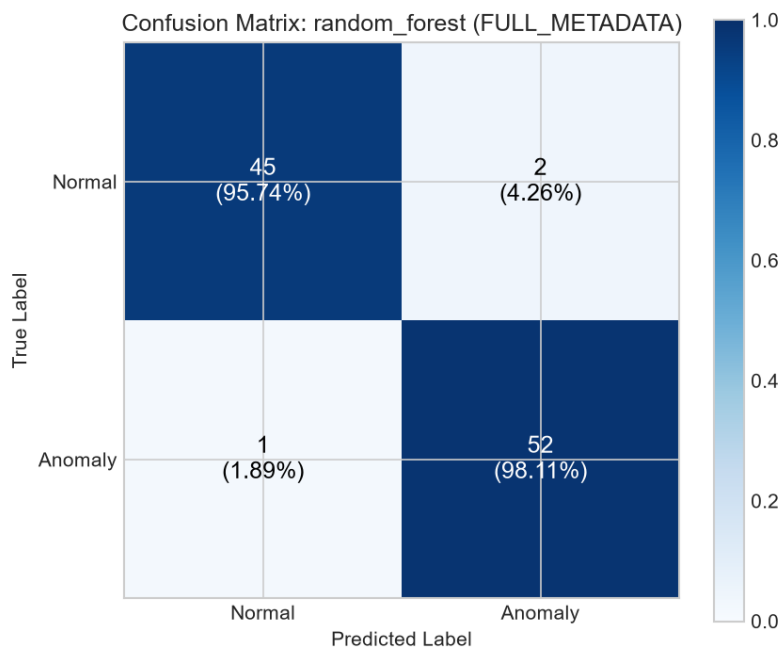
- Majority baseline: always predicts the anomaly class (the majority here), so recall is 1.0 and the false positive rate is 1.0. Its F1 of 0.6928 is the floor a real model must beat.
- Logistic Regression: F1 0.5941, FPR 0.3830. A linear decision boundary does not separate the classes well on this data - it does not beat the baseline.
- Random Forest: F1 0.9720, FPR 0.0426 - the best result, with 2 false positives and 1 false negative out of 100 rows.

What changed between the feature sets

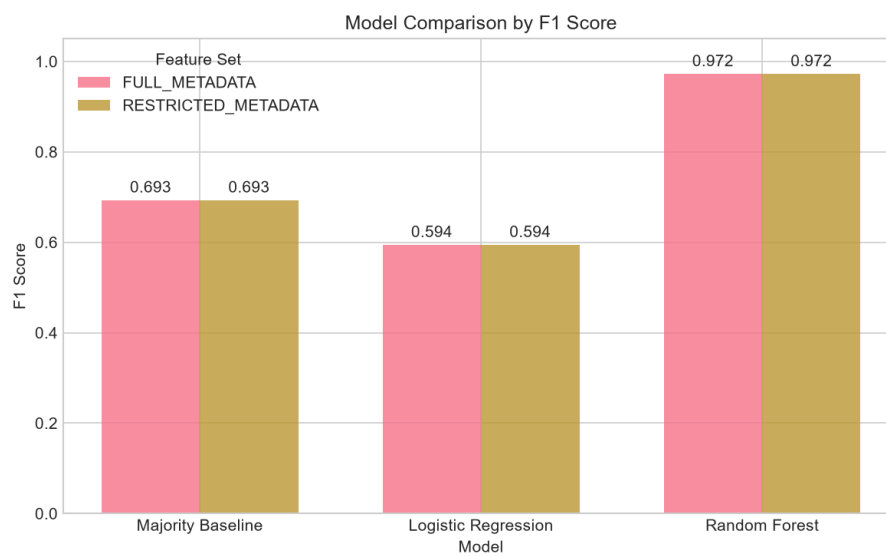
On this run the two feature sets produced identical classification results for all three models: Random Forest reached F1 0.9720 on both, and Logistic Regression reached F1 0.5941 on both. Only the probability-based metrics moved, by less than 0.001 (ROC AUC 0.9940 vs 0.9948 for the Random Forest). For this sample, reducing 37 features to 12 changed nothing measurable - the extra features carry redundant signal at this scale.

Honest reading: this is a 100-row smoke test, not evidence that restricted features are always sufficient. The secondary research question can only be answered on the full 250,000-row dataset.

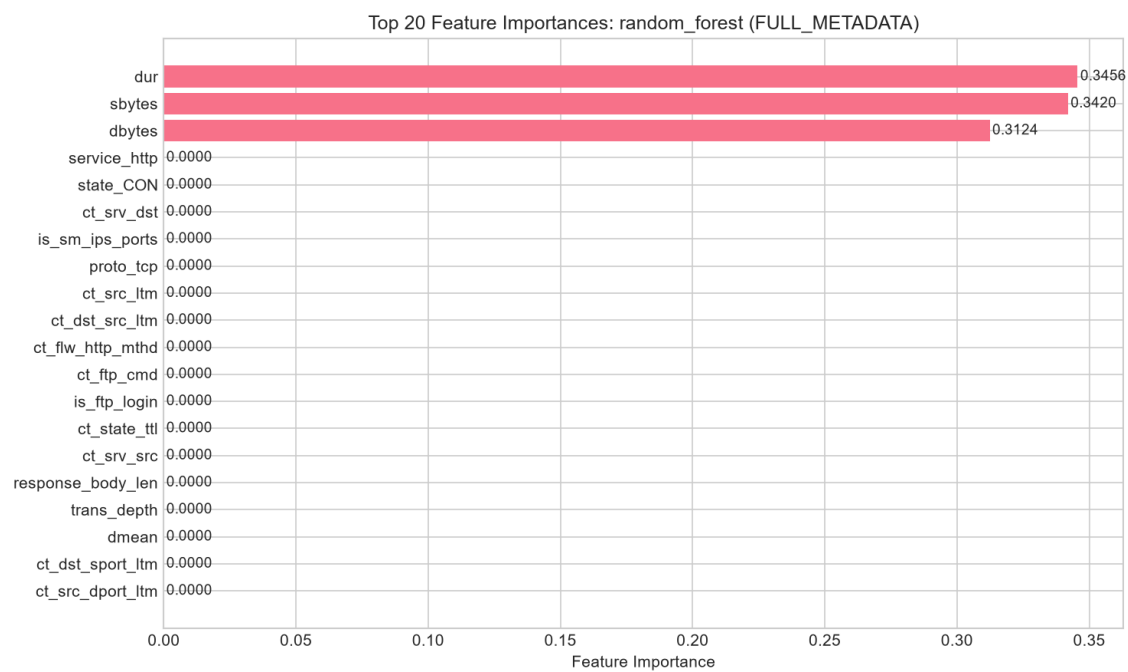
Figures



Confusion matrix - Random Forest, FULL_METADATA



Model comparison by F1 score

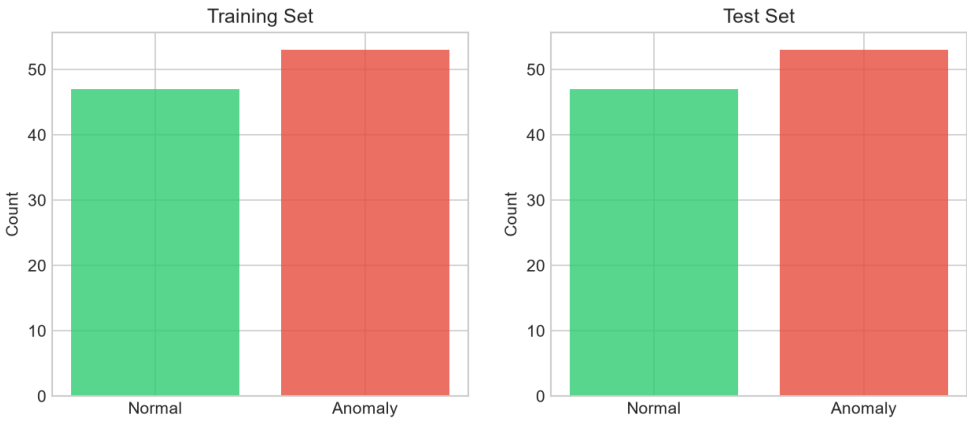
*Model comparison by false positive rate**Feature importance - Random Forest, FULL_METADATA*

47 (47.0%)

53 (53.0%)

47 (47.0%)

53 (53.0%)



Class distribuition - train vs test

7. Reproducing the experiment

A fresh clone reproduces everything with four commands:

```
git clone https://github.com/TMStroo/Privacy-Preserving-Network-Anomaly-Detection
cd Privacy-Preserving-Network-Anomaly-Detection
pip install -r requirements.txt && pip install -e .
python scripts/create_sample_data.py      # or drop the real UNSW-NB15 CSVs in
data/raw/
python -m privacy_preserving_nad.pipeline
pytest tests/ -v
```

The pipeline command runs validation, preprocessing, training, evaluation and plotting in one pass. Individual stages can be run with `--step` `validate`, `preprocess`, `train`, `evaluate` or `plot`. All knobs - dataset paths, feature lists, target column, seed, model hyperparameters and output directories - live in `configs/config.yaml`.

Determinism: random seeds are fixed (42) for numpy, python's random and every estimator, the train/test split is the dataset's official one, and preprocessing is fit only on training data. Repeated runs on the same inputs produce the same metrics.

Continuous integration runs the test suite and a pipeline smoke test on Python 3.9, 3.10 and 3.11, using the generated sample data so no large download is needed.

Project structure

<code>configs/config.yaml</code>	all experiment configuration
<code>data/raw/</code>	raw dataset (gitignored)
<code>data/processed/</code>	processed sets (gitignored)
<code>src/privacy_preserving_nad/</code>	
<code>data/</code>	download, validate, preprocess
<code>features/</code>	metadata feature policy
<code>models/</code>	baselines, training, inference
<code>evaluation/</code>	metrics and plots
<code>pipeline.py</code>	orchestration entry point
<code>tests/</code>	40 pytest tests
<code>notebooks/</code>	exploratory analysis
<code>results/metrics/, figures/</code>	generated outputs
<code>docs/</code>	this report
<code>.github/workflows/ci.yml</code>	CI pipeline

8. Limitations

- Dataset age: UNSW-NB15 dates from 2015 and predates widespread TLS 1.3, QUIC and

encrypted SNI; modern traffic looks different.

- Dataset-specific behavior: attacks are synthetic, and generators do not reproduce determined real-world adversaries.
- Distribution shift: the split is temporal within the dataset; production traffic will differ further.
- Feature availability: fields like tcprrt, synack and ackdat require visibility of the TCP handshake, which not every collection point has.
- Binary simplification: distinct attack categories are collapsed into one anomaly class, hiding per-attack differences.
- Sample-scale numbers: the results in this report come from 100-row sample data and are not benchmark results.
- Scope of privacy: 'privacy-preserving' means no payload is read. Metadata alone can still identify and profile users; this project proves no formal privacy guarantee.

Future directions

- Re-run on the full UNSW-NB15 split and on newer datasets such as CIC-IDS2017.
- Test adversarial robustness: can an attacker shuffle metadata to evade these detectors?
- Study temporal concept drift in metadata distributions.
- Extend from binary detection to multi-class attack attribution.
- Evaluate feature attribution stability and deploy as a NetFlow/IPFIX consumer.

License

MIT License. See the LICENSE file in the repository.